Why is MQTT preferable to alternative communication protocols for IoT monitoring and control systems, and how can its characteristics justify the architecture of a smart greenhouse system?

1. Introduction

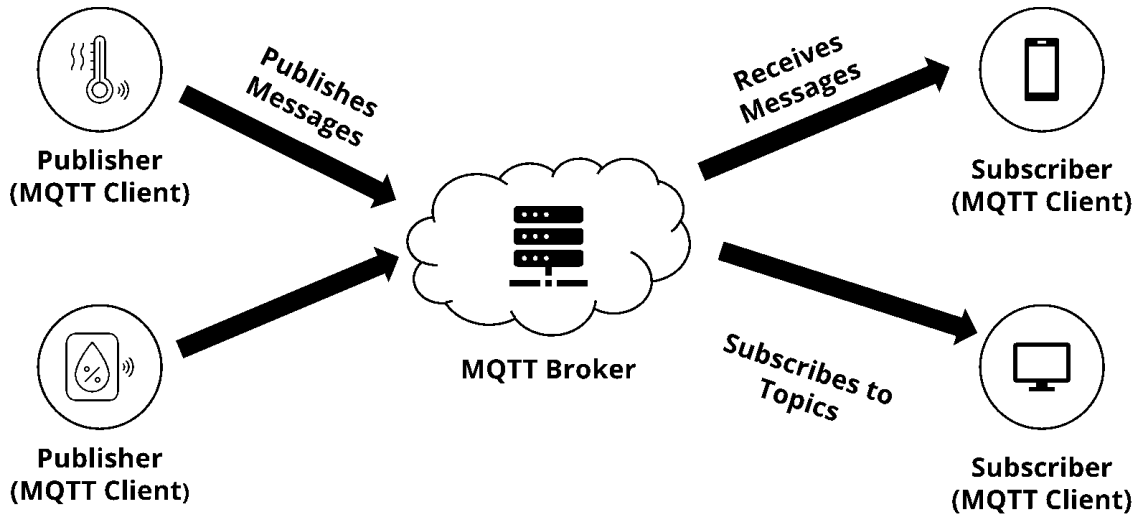
The Internet of Things (IoT) enables physical devices to collect and exchange data over a network. In monitoring and control systems, choosing an appropriate communication protocol is important because it affects communication efficiency, reliability, and the way different system components interact.

Several protocols, including MQTT, HTTP, and CoAP, can be used for IoT communication, but their characteristics make them suitable for different situations.

This paper investigates what communication protocol best suits the monitorning and control requirements of the proposed smart greenhouse.

2. What is MQTT?

MQTT is a lightweight, publish–subscribe, machine-to-machine network protocol for message queueing/message queueing services. It is designed for connections with remote locations that have devices with resource constraints or limited network bandwidth, such as in the Internet of things (IoT). It must run over a transport protocol that provides ordered, lossless, bi-directional connections, typically TCP/IP.



MQTT Fundamentals

The MQTT protocol defines two types of network entities: a message broker and a number of clients

1. Broker - server that receives messages from publishing clients and then routes the messages to the appropriate destination clients.

2. Client - any device (from a microcontroller up to a full-fledged server) that runs an MQTT library and connects to an MQTT broker over a network. Clients are devided into 2 types – publishers and subscribers.

N.B! A client can be both a publisher and subscriber!

3. Publisher - a client device or software application that sends messages to an MQTT Broker under a specific topic

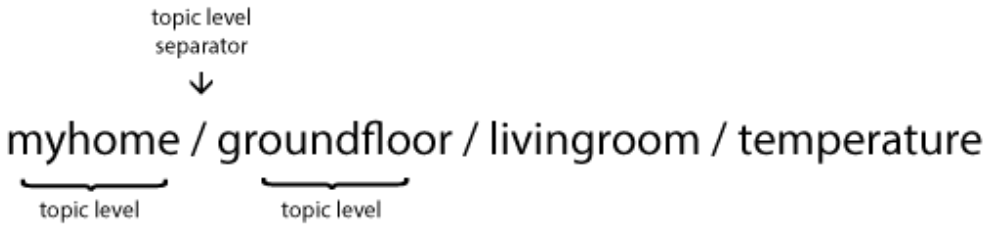
4. Subscriber - a client device or application that connects to an MQTT broker and listens for messages published to specific topics

Information is organized in a hierarchy of topics. When a publisher has a new item of data to distribute, it sends a control message with the data to the connected broker. The broker then distributes the information to any clients that have subscribed to that topic. The publisher does not need to have any data on the number or locations of subscribers; and subscribers, in turn, do not have to be configured with any data about the publishers.

5. Topics - UTF-8 string that filters messages for a connected client. A topic consists of one or more levels separated by a forward slash (topic level separator). The UTF-8 strings act as identifiers for messages, enabling a publish/subscribe system where publishers send messages to specific topics and subscribers can subscribe to those topics to receive relevant messages.

N.B! - In comparison to a message queue, MQTT topics are very lightweight. The client does not need to create the desired topic before they publish or subscribe to it. The broker accepts each valid topic without any prior initialization.

A



6. Quality of service (QoS) - the description or measurement of the overall performance of a service.

How does it work with MQTT?

Each connection to the broker can specify a QoS measure. These are classified in increasing order of overhead:

6.1. At most once – the message is sent only once and the client and broker take no additional steps to acknowledge delivery (fire and forget)

6.2. At least once – the message is re-tried by the sender multiple times until acknowledgement is received (acknowledged delivery)

6.3. Exactly once – the sender and receiver engage in a two-level handshake to ensure only one copy of the message is received (assured delivery).

7. Retained messages - A retained message is a normal MQTT message with the retained flag set to true. The broker stores the last retained message and the corresponding quality of service (QoS) for the selected topic. Each client that subscribes to a topic pattern that matches the topic of the retained message receives the retained message immediately after they subscribe. The broker stores only one retained message per topic.This allows new subscribers to a topic to receive the last retained value rather than waiting for the next update from a publisher. It acts as a pinned message.

If a broker receives a message on a topic for which there are no current subscribers, the broker discards the message unless the publisher of the message designated the message as a retained message

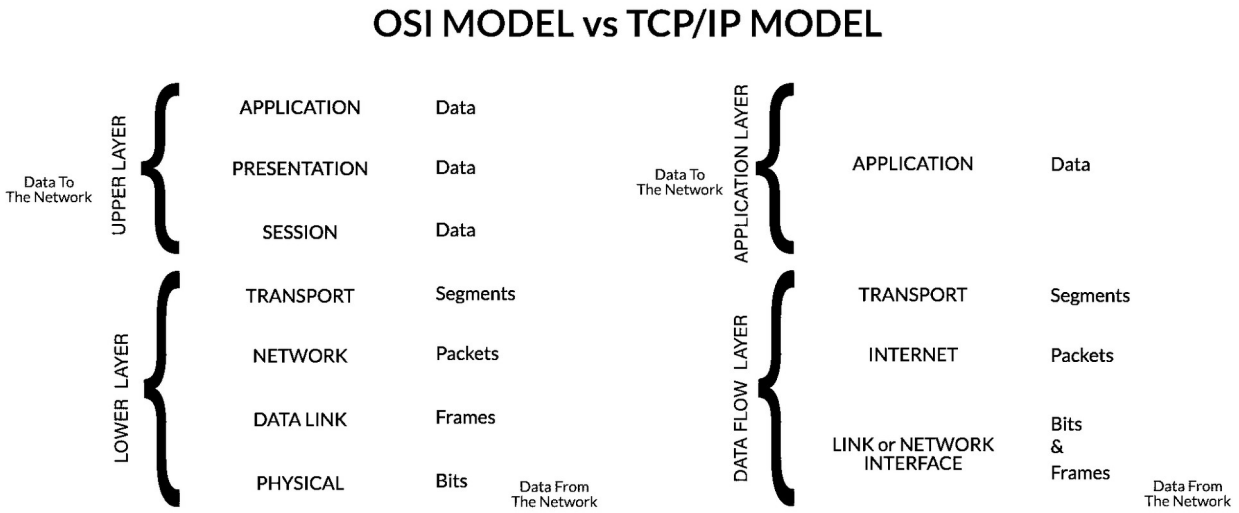
8. What is the TCP/IP

8.1. The Internet protocol suite, commonly known as TCP/IP, is a framework for organizing the communication protocols used in the Internet and similar computer networks according to functional criteria. The foundational protocols in the suite are the Transmission Control Protocol (TCP), the User Datagram Protocol (UDP), and the Internet Protocol (IP). Unlike the OSI Model, which is a theoretical reference framework used for design, analysis and education, the TCP/IP is a practical, functional standard that powers the global internet. It only has 4 layers, while the OSI Model has 7.

8.2. The Transmission Control Protocol (TCP) provides reliable, ordered, and error-checked delivery of a stream of octets (bytes) between applications running on hosts communicating via an IP network

8.3. User Datagram Protocol (UDP) is a connectionless, "fire-and-forget" protocol that sends data packets without establishing a connection or waiting for acknowledgments (unlike TCP). It does not reorder or retransmit lost data, resulting in a fixed, lightweight 8-byte header. It does NOT work with MQTT!

8.4. The Internet Protocol (IP) is the foundational set of rules that governs how data packets are addressed, sent, and routed across computer networks. IP has the task of delivering packets from the source host to the destination host solely based on the IP addresses in the packet headers. For this purpose, IP defines packet structures that encapsulate the data to be delivered. It also defines addressing methods that are used to label the datagram with source and destination information. In short it acts as the technology that routes the packets troughout the internet.



2. MQTT Advantages and Disadvantages

Advantages:

1. Reduced complexity

MQTT uses a minimal, binary header format (the smallest possible packet is just 2 bytes), binary encoding and a decoupled* Publish/Subscribed architecture, eliminating the heavy parsing overhead and complex connection management found in other protocols like HTTP.

---

*decoupling - separating two or more systems, activities, or economic factors so that they no longer move, grow, or function together. Or in other words, removing the dependencies between components that send data (publishers) and components that receive data (subscribers).

2. Elimination of “Polling”

Polling is a technique where a client/program repeatedly asks a server/database/hardware device at regular intervals to check if new data is available or if a condition has changed. It is simple and predictable, but it wastes resources and introduces latency.

MQTT eliminates polling by using a push-based pub/sub architecture in which the client establishes a persistent connection and waits for the broker to push updates automatically

3. Ideal for low energy consumption

MQTT is ideal for low energy consumption, because its binary design and minimal packet headers reduce the amount of time a device’s wireless radio needs to stay powered on.

MQTT can also use a Keep-Alive mechanism* to maintain connection state over long periods without flooding the network with redundant traffic.

MQTT’s configurable QoS can also bypass delivery-acknowledgement handshakes when power conservation is more critical than guaranteed message receipt

---

*Keep Alive is a feature of the MQTT protocol that allows an MQTT client to maintain its connection with a broker by sending regular control packets called PINGREQ to the broker

4. Lightweight

The protocol overhead of MQTT is extremely small and the protocol was overall designed to send small messages (256MB max). This, along with the binary design, makes it extremely lightweight

Disadvantages:

1. Single point of failure

The MQTT protocols uses centralized routing. This means that all publishers and subscirbers route their messages trough a central server. Due to this, if this server (aka the broker) goes offline, communication across the entire system stops.

2. Lack of scalability

The Single Point Of Failure (SPOF) introduces another problem. It’s the lack of scalability. Because of the centralized broker dependence and the maintainance of persistent TCP connections and session states for every connected client (consuming RAM and CPU), horizontal scaling becomes a problem.

Disadvantage 1 and 2 can be overcomed by the concept of clustering.

MQTT clustering means grouping connected MQTT brokers that work together as a single, unified messaging layer in order to provide high availability, fault tolerance, and horizontal scalability. A grouping of brokers in MQTT is called a cluster.

3. Works only on bidirectional, transport protocols, typically TCP

The MQTT protocol does NOT support the UDP protocol and relies strictly on the TCP transport layer, which introduces higher memory and processing overhead, battery drain and head-of-line blocking*.

This can be overcomed by adjusting the QoS parameter. By setting it to “At most once” the message is sent only once and no further steps for acknowledging delivery are taken. This mimics the UDP protocol, but it is still using the TCP and doesn’t remove the hed-of-line blocking

---

*Head-Of-Line Blocking is a performance bottleneck where a queue of packets or data is stalled by the packet sitting right at the front of it, forcing everything behind it to wait.

3. Comparison with other protocols

Feature	**MQTT**	**HTTP**	**CoAP**
Model	Publish / Subscribe (via Broker)	Request / Response (Client-Server)	Request / Response (REST-like, Peer-to-Peer capable)
Transport Layer	TCP (persistent connection)	TCP / HTTP/3	UDP (with confirmable/non-confirmable options)
Overhead	Very low (2-byte fixed header)	High (verbose text headers, cookies)	Very low (binary header, small size)
Security	TLS / SSL (handled by developer)	TLS 1.3 / HTTPS	DTLS (Datagram TLS)
Quality of Service (QoS)	Levels 0, 1, and 2 (guaranteed delivery options)	Relies on TCP reliability	Confirmable vs. Non-confirmable messages
Best Used For	Real-time sensor telemetry, pub-sub messaging, unstable/cellular networks	Standard web APIs, heavy data transfers, cloud integration	Extremely constrained microcontrollers, low-power sensor mesh networks

4. Why MQTT fits my greenhouse project best?

MQTT fits the greenhouse project best, because of it’s architecture, low power cost, low overhead and high speed.

In my case, I need a central device, that collects data from different sensors and displays this data, sends it to an external database and controls actuators based on it. I’m not choosing HTTP because of its high overhead and heavy dta transfers. This is an obvious choice. The other option would be CoAP, which has similar features compared with MQTT, but the main problem with it is the architecutre, which doesn’t satisfy my needs.

6. Sources

MQTT Wikipedia page:
https://en.wikipedia.org/wiki/MQTT

TCP/IP wikipedia page:
https://en.wikipedia.org/wiki/Internet_protocol_suite

TCP wiki:
https://en.wikipedia.org/wiki/Transmission_Control_Protocol

UDP wiki:
https://en.wikipedia.org/wiki/User_Datagram_Protocol

IP wiki:
https://en.wikipedia.org/wiki/Internet_Protocol

OSI model wiki:
https://en.wikipedia.org/wiki/OSI_model

MQTT essentials:
https://mqtt.org/
https://www.hivemq.com/mqtt/

HTTP:
https://en.wikipedia.org/wiki/HTTP

COAP:
https://en.wikipedia.org/wiki/Constrained_Application_Protocol